

ABM BUSINESS LOGIC BIBLE

SECTION 4 — INTEGRATIONS, APIs & FAILURE MODES

Full 5-Whys Excavation · 11 Integration Domains · The External Boundary of the ABM Engine

Sections 1–3 defined the engine's principles, entities and modules. Section 4 defines its **boundary with the outside world** — every external system it reads from or writes to, the service contracts it exposes, and the failure architecture that keeps it from corrupting the system of record. The headline domain, **INT-SIG**, is a full Account Intelligence Acquisition Engine that automates the proven BD-intelligence methodology and resolves the connection path back to Decimal.

Domains	11	5-Whys chains	18
New rules	~111	Headline	INT-SIG / SIG-PATH

#	Domain	Scope	Rules
4.1	INT-SIG	Account Intelligence Acquisition Engine (8 sub-streams) + SIG-PATH	34
4.2	INT-ENR	Enrichment (Apollo/Clearbit/Clay) — fallback chain, cost gating	7
4.3	INT-LIN	Heyreach LinkedIn — platform-policy limits, ban-risk circuit breaker	7
4.4	INT-EML	Email + deliverability — SPF/DKIM/DMARC, warmup gating	8
4.5	INT-CRM	HubSpot sync — idempotency, authority split, conflict resolution	6
4.6	INT-SLK	Slack notifications — brief contract, rate limiting	4
4.7	INT-ANL	Metabase analytics — read-replica, no write-back	4
4.8	INT-ORC	n8n orchestration — no-business-logic boundary, idempotent steps	7
4.9	API	Service contracts — 5 endpoints, auth, versioning, rate limits	10
4.10	FAIL	Retry / DLQ / circuit-breaker matrix	9
4.11	OBS	Observability + PDPL / security / compliance	7

4.1 INT-SIG — ACCOUNT INTELLIGENCE ACQUISITION ENGINE

INT-SIG is not a feed reader. It continuously produces, per target account, a **living Master Intelligence Dossier** and resolves the connection path back to Decimal — automating the BD methodology proven on SNB, Riyadh Bank and Al Rajhi. Eight sub-streams harvest into one account intelligence graph; the headline stream SIG-PATH is the acquisition target, since a signal with no warm path is unactionable.

Sub-stream	Acquires (Riyad Bank example)	Cadence
SIG-NEWS	News, announcements, press releases, public circulars	Hourly
SIG-EVENT	Events, seminars, conferences, awards, speaking slots	Daily
SIG-FIN	Financial position, quarterly results, annual report, ratings	On-publish + quarterly
SIG-REG	SAMA/CMA regulatory circulars, licences, compliance moves	Daily
SIG-EXEC	Full C-suite dossier: bio, tenure, public speeches/quotes, moves	Weekly + event
SIG-VENDOR	Vendor relationships + activity (Efignce at Riyad) + gap	Monthly + event
SIG-SUBS	Subsidiaries and their activity	Monthly + event
SIG-PATH	Connection paths to Decimal (LinkedIn degree, mutual, shared node)	Continuous

SIG-PATH — connection-to-Decimal (resolved against the full set)

Priority ladder: P1 LinkedIn 1st-degree (Decimal employee ↔ target) > P2 mutual connection via a Decimal person > P3 shared node (ex-employer / alumni / vendor / board) > P4 generalized connector bridge (KPMG/Mastercard/Deloitte/Tarabut/Fintech-Saudi) > P5 cold email pattern. Resolved against **both** Decimal's ingested employee network and the generalized connector registry.

Hard-coded tradecraft: cross-group power-node detection (one exec across ≥2 entities), IT-subsiary gatekeeper sequencing (route through the EJADA-equivalent before C-suite), path decay TTL, executive-departure-as-dual-signal, and strict anti-fabrication (never invent a path or a contested name).

INT-SIG carries 34 rules (SIG-PATH 10 · SIG-EXEC 5 · SIG-VENDOR 4 · SIG-SUBS 2 · SIG-NEWS 2 · SIG-REG 2 · SIG-FIN 1 · SIG-EVENT 1, plus the consolidated index). Full 5-Whys for all eight sub-streams is documented in the standalone Checkpoint 4.1 PDF and is incorporated here by reference to keep this unified document navigable.

4.2 INT-ENR — Enrichment — Apollo · Clearbit · Clay

Enrichment resolves accounts and buying committees into verified, reachable contacts. The FSD mandates verified email enrichment only; INT-ENR specifies the provider chain, cost gating and conflict-merge that deliver it.

Why #1: Why use a fallback chain across three providers rather than one?

Because no single enrichment source covers the KSA banking ecosystem completely; a verified email missing in Apollo may exist in Clearbit or be inferable via Clay. A single provider produces unreachable contacts and bounce risk.

Why #2: Why gate enrichment on cost / eligibility rather than enrich everything?

Because enrichment calls cost money and rate-limit budget; enriching COLD/unreachable accounts wastes both. Enrichment is a scarce resource allocated by tier and reachability, consistent with Scarcity Economics.

Why #3: Why is verified-email-only non-negotiable?

Because an unverified email both bounces (damaging sender reputation, which INT-EML depends on) and pollutes the contact state machine with false REACHABLE statuses.

Why #4: Why specify a conflict-merge policy across providers?

Because providers disagree (different titles, stale roles); without a deterministic merge the engine stores contradictory contact facts and may address the wrong person.

Why #5: Why feed enrichment freshness back into scoring?

Because a stale contact record silently degrades reachability; freshness must be visible to scoring so an account isn't prioritized on the basis of a contact that no longer exists.

Root cause: Enrichment is a cost-gated, verified-only, fallback-chained resolution step whose freshness and conflicts must be explicit, never silently trusted.

New rules

Rule ID	Specification
INT-ENR-01	Provider fallback chain is ordered Apollo → Clearbit → Clay; resolution stops at the first verified result. Order is configurable but the verified-only gate is not.
INT-ENR-02	Only verified emails make a contact email-eligible; unverified → LinkedIn-only outreach path (per FSD ENR decision logic).
INT-ENR-03	Enrichment is gated by tier + reachability: HOT/WARM and path-reachable accounts enrich first; COLD/unreachable are deferred.
INT-ENR-04	Cross-provider conflict-merge: most-recent + highest-source-trust wins; unresolved title/role conflicts are flagged ~ and excluded from automated send.
INT-ENR-05	Every enriched record carries source + timestamp + freshness TTL; on expiry the contact is re-enriched before reuse.
INT-ENR-06	Buying-committee coverage is required: single-threaded enrichment (one contact per account) is flagged as insufficient coverage to scoring.
INT-ENR-07	Enrichment cost/volume is budgeted and logged; spend per account is capped and visible to the Scarcity (capital) layer.

4.3 INT-LIN — LinkedIn Automation — Heyreach

Heyreach executes LinkedIn connection requests and DMs. LinkedIn aggressively bans automation that breaches its platform policy, so INT-LIN is governed by hard caps and a ban-risk circuit breaker.

Why #1: Why hard-cap daily LinkedIn actions per sending account?

Because LinkedIn detects and bans accounts that exceed human-plausible action volumes; a banned account destroys an irreplaceable warm channel and any SIG-PATH P1 edges anchored on it.

Why #2: Why a circuit breaker rather than just a rate limit?

Because ban risk is non-linear: warning signs (restricted views, captcha challenges, withdrawal spikes) precede a ban. A breaker halts the account on early signals before the irreversible ban, which a static rate limit cannot.

Why #3: Why coordinate LinkedIn with email rather than run them independently?

Because simultaneous multi-channel contact reads as spam to the prospect and damages experience; the FSD mandates coordinated, not concurrent, channel use.

Why #4: Why tie LinkedIn sending accounts to SIG-PATH?

Because the strongest path (P1/P2) often runs through a specific Decimal person's LinkedIn; the sending account must match the warm path, not be assigned round-robin.

Why #5: Why treat a connection-accept as a state transition, not just a metric?

Because acceptance changes what outreach is permitted (DM eligible) and feeds reply detection / handoff; it must update the contact state machine, not merely increment a counter.

Root cause: LinkedIn automation must operate inside platform-policy-safe caps with an early-warning circuit breaker, warm-path-matched sending accounts, and coordinated (never concurrent) channel timing — because a banned account is an irreversible loss of warm capital.

New rules

Rule ID	Specification
INT-LIN-01	Per-account daily caps on connection requests + DMs + profile views are enforced below LinkedIn's human-plausible thresholds; caps are configurable downward only.
INT-LIN-02	Ban-risk circuit breaker: on early-warning signals (captcha challenge, view restriction, accept-rate collapse, withdrawal spike) the account is auto-paused and flagged for human review.
INT-LIN-03	Channels are coordinated, not concurrent: email and LinkedIn touches to the same contact are spaced per sequence design; simultaneous same-day multi-channel contact is blocked.
INT-LIN-04	Sending-account assignment matches the resolved SIG-PATH where a P1/P2 Decimal-person path exists; round-robin only for P4/P5 cold paths.
INT-LIN-05	Connection-accept and DM-reply update the contact state machine (→ OPENED/REPLIED) and trigger reply-detection + handoff logic.
INT-LIN-06	Account warmup: new sending accounts ramp action volume gradually before reaching standard caps.
INT-LIN-07	A banned/restricted account immediately emits SIG-PATH path-break events for every P1 edge it anchored.

4.4 INT-EML — Email Execution & Deliverability

The FSD treats deliverability as a strategic layer, not a technical afterthought: enterprise outreach succeeds or fails on inbox placement and sender reputation. INT-EML specifies the infrastructure and gating that protect it.

Why #1: Why mandate SPF, DKIM and DMARC before any sending?

Because without authentication, enterprise mail filters route outreach to spam regardless of content quality; the entire personalization investment is wasted at the gateway.

Why #2: Why gate sending on domain warmup?

Because a cold domain sending at volume is the strongest spam signal; warmup builds the sender reputation that inbox placement depends on.

Why #3: Why throttle volume and rotate sending identities?

Because spikes from a single identity trip volume-based spam heuristics; low-volume, rotated, high-relevance sending is the FSD's mandated posture (low volume / high personalization, never mass blast).

Why #4: Why feed bounce/spam telemetry back as a reputation signal?

Because reputation degrades silently; without a feedback loop the engine keeps sending into a damaged reputation, compounding the damage. Telemetry must gate further sending.

Why #5: Why couple email reputation to enrichment (INT-ENR) verification?

Because the fastest way to destroy reputation is sending to unverified addresses that bounce; verified-only enrichment is a precondition of deliverability, linking the two domains.

Root cause: Email is a reputation-managed channel: authenticated, warmed, volume-throttled, rotation-balanced, and reputation-feedback-gated — with verified-only recipients as a hard precondition.

New rules

Rule ID	Specification
INT-EML-01	SPF, DKIM and DMARC must be configured and validated per sending domain before that domain is permitted to send.
INT-EML-02	Domain warmup is mandatory: new domains ramp volume on a schedule and cannot send at full volume until warmed.
INT-EML-03	Per-domain volume throttles + sending-identity rotation enforce a low-volume, high-relevance posture; mass blasting is structurally blocked.
INT-EML-04	Bounce, complaint and spam-placement telemetry feed a per-domain reputation score; below a floor, the domain is auto-paused from sending.
INT-EML-05	Only INT-ENR-verified emails may be sent to; unverified addresses are never sent email (LinkedIn-only path).
INT-EML-06	Reply received pauses the sequence and updates contact state (→ REPLIED) per FSD EXEC-002.
INT-EML-07	Duplicate-outreach prevention: the same contact is never sent the same touch twice across workflows (FSD EXEC-003).
INT-EML-08	Arabic / English send selection follows the contact's resolved language preference; deliverability rules apply identically to both.

4.5 INT-CRM — CRM Synchronization — HubSpot

After positive engagement, sales takes over and needs full context in the system of record. INT-CRM specifies a sync that cannot corrupt that system of record — the failure mode that destroys trust in the whole engine.

Why #1: Why sync engagement history to the CRM at all?

Because a cold open from sales destroys a warm signal; sales needs the full engagement context to take over without re-pitching, and the CRM is the system they already trust.

Why #2: Why split authority between ABM and CRM rather than one source of truth?

Because dual authority over the same field causes write-conflicts; ABM is authoritative for signals/scores, CRM for deal stage/owner. No field may be authoritative in both.

Why #3: Why must sync be idempotent?

Because retries (network, queue) re-deliver the same event; without idempotency the CRM accumulates duplicate tasks/activities, polluting the record sales relies on.

Why #4: Why route failed syncs to a dead-letter queue rather than retry forever or drop?

Because a silently dropped sync leaves ABM and CRM disagreeing on account state — and a disagreement can cause automation to re-engage an account sales has already closed. A DLQ makes failures visible and recoverable.

Why #5: Why have CRM deal-stage win conflicts and pause automation?

Because the human-owned deal stage reflects ground truth that the engine cannot see; when they conflict, the engine must defer to the human and stop automated outreach to avoid contradicting sales.

Root cause: CRM sync must be idempotent, authority-split, conflict-resolved in favour of the human-owned deal stage, and dead-lettered on failure — because corrupting the system of record is the one failure that discredits the engine.

New rules

Rule ID	Specification
INT-CRM-01	Sync is idempotent on (account_id, event_hash); re-delivery of an already-synced event is a no-op.
INT-CRM-02	Authority split: ABM is authoritative for signals/scores/engagement; CRM is authoritative for deal stage/owner. No field is dual-authoritative.
INT-CRM-03	Failed syncs route to a dead-letter queue with full payload + error; never a silent drop.
INT-CRM-04	On state conflict, CRM deal-stage wins and automated outreach to that account is paused pending human review.
INT-CRM-05	Reply / positive engagement triggers: pause automation → assign owner → generate AI meeting brief → Slack notify → create CRM task (FSD handoff sequence).
INT-CRM-06	CRM credentials are encrypted secrets; sync runs under least-privilege scopes (FSD security architecture).

4.6 INT-SLK — Notifications — Slack

Slack delivers the human-handoff alert and AI meeting brief to the sales team at the decisive moment. INT-SLK specifies a reliable, rate-safe, well-formed notification contract.

Why #1: Why push handoff notifications to Slack rather than only the CRM task?

Because timing is the competitive weapon: a positive reply decays fast, and a Slack ping reaches the owner in real time where a CRM task waits to be noticed.

Why #2: Why define a strict brief payload contract?

Because an underspecified alert forces the rep back into research, losing the time advantage; the brief must carry enough grounded context (account, signal, path, suggested angle) to act immediately.

Why #3: Why rate-limit Slack notifications?

Because alert floods cause the team to ignore them (alert fatigue), defeating the purpose; notifications must be deduplicated and prioritized.

Why #4: Why require delivery confirmation?

Because a silently failed notification means a hot handoff is never picked up; delivery must be confirmed or the alert escalated/retried.

Root cause: Slack notifications must be real-time, contract-formed with a grounded brief, rate-limited against fatigue, and delivery-confirmed so no hot handoff is silently lost.

New rules

Rule ID	Specification
INT-SLK-01	Handoff notification fires in real time on the engagement trigger, carrying the AI meeting brief.
INT-SLK-02	Brief payload contract: account, triggering signal, resolved warm path/owner, and suggested angle — all grounded in INT-SIG facts.
INT-SLK-03	Notifications are deduplicated and priority-ranked; volume is rate-limited to prevent alert fatigue.
INT-SLK-04	Delivery is confirmed; an unconfirmed/failed notification is retried then escalated, never silently dropped.

4.7 INT-ANL — Analytics — Metabase

Analytics are not only reporting — the FSD uses them for system learning, signal optimization and prompt refinement. INT-ANL specifies a safe, read-only analytics boundary.

Why #1: Why connect Metabase to a read replica, never the primary?

Because analytical queries are heavy and unpredictable; running them on the primary risks degrading the live ingestion/scoring pipeline.

Why #2: Why forbid analytics write-back into operational tables?

Because analytics is a consumer, not a producer, of operational truth; letting dashboards write back creates an uncontrolled path that can corrupt scoring inputs.

Why #3: Why define an explicit KPI contract?

Because the analytics layer feeds the optimization loop (scoring weights, prompt refinement); ambiguous metric definitions silently mis-tune the engine.

Why #4: Why route optimization changes through a controlled loop rather than direct dashboard edits?

Because changing scoring weights or prompts is a model-governance action; it must be reviewed and versioned, not applied live from a BI tool.

Root cause: Analytics is a read-only, contract-defined consumer whose insights feed a governed optimization loop — never a direct write path into the operating engine.

New rules

Rule ID	Specification
INT-ANL-01	Metabase reads from a read replica only; no analytical query touches the primary operational database.
INT-ANL-02	Analytics has no write access to operational tables; it is strictly read-only.
INT-ANL-03	KPI definitions are contracted and versioned (signal, persona, messaging, conversion performance) so the optimization loop tunes on stable metrics.
INT-ANL-04	Optimization outputs (weight/prompt changes) flow through a governed, reviewed, versioned loop — not applied directly from the BI layer.

4.8 INT-ORC — Orchestration — n8n

n8n is the orchestration layer, event router and integration engine. The FSD is explicit: n8n must NOT contain business logic, scoring logic, or AI reasoning — those belong in APIs/services. INT-ORC enforces that boundary.

Why #1: Why keep business/scoring/AI logic out of n8n?

Because logic embedded in workflows is untestable, unversioned and invisible to governance; the same rule could drift across workflows. Logic belongs in versioned services with a single source of truth.

Why #2: Why make every workflow step idempotent?

Because n8n retries and re-runs steps on failure; non-idempotent steps double-send outreach or double-write CRM records on retry.

Why #3: Why group workflows (signal / intelligence / execution / analytics)?

Because grouping isolates failure domains and clarifies ownership; a failure in execution workflows shouldn't halt signal ingestion.

Why #4: Why standardize the

Trigger→Validation→Decision→Execution→Logging→State-update pattern?

Because a uniform step pattern guarantees every workflow validates inputs, logs, and updates state consistently — the observability and recovery layers depend on that uniformity.

Why #5: Why route all external calls through n8n rather than service-to-service?

Because centralizing integration in the orchestration layer gives one place for retry, rate-limit and observability policy, instead of scattering it across services.

Root cause: n8n is a logic-free orchestration layer of idempotent, uniformly-patterned, grouped workflows — the integration and retry/observability choke point, with all business logic delegated to versioned services.

New rules

Rule ID	Specification
INT-ORC-01	n8n contains orchestration only: no business logic, no scoring logic, no AI reasoning. These live in versioned APIs/services.
INT-ORC-02	Every workflow step is idempotent; retries and re-runs cannot double-send outreach or double-write records.
INT-ORC-03	Workflows are grouped into Signal / Intelligence / Execution / Analytics domains with isolated failure boundaries.
INT-ORC-04	Every workflow follows the standard pattern: Trigger → Validation → Decision → Execution → Logging → State update.
INT-ORC-05	All external integration calls route through n8n so retry, rate-limit and observability policy are applied in one place.
INT-ORC-06	Workflow execution, decisions and errors are logged to the observability layer (OBS) on every run.
INT-ORC-07	n8n calls services over the defined API contracts (4.9); it never reaches into service databases directly.

4.9 API — Service Contracts & Interface Design

The engine exposes modular APIs (FastAPI) for scoring, enrichment, AI generation, CRM sync and analytics, enabling modular orchestration and future frontend support. 4.9 specifies the contracts.

Why #1: Why expose modular APIs rather than a monolith?

Because modular services allow independent scaling, isolation and a future frontend; they also give n8n clean contracts to orchestrate against without embedding logic.

Why #2: Why version every API contract?

Because the engine and its orchestration evolve independently; unversioned contracts break workflows silently on change. Versioning lets producers and consumers evolve safely.

Why #3: Why authenticate and scope every endpoint?

Because these services handle enterprise contact data and banking context; an unauthenticated or over-scoped endpoint is a data-exposure path (PDPL exposure).

Why #4: Why rate-limit service endpoints internally?

Because a runaway workflow or retry storm can overwhelm a service (especially AI generation and enrichment, which are costly); internal rate limits protect availability and cost.

Why #5: Why define explicit request/response schemas and error contracts?

Because the retry/DLQ layer (4.10) keys its recovery strategy on error type; without a stable error contract the failure architecture cannot distinguish a retryable timeout from a permanent rejection.

Root cause: Every service is a versioned, authenticated, scoped, rate-limited contract with explicit request/response and error schemas — the stable surface the orchestration and failure layers depend on.

New rules

Rule ID	Specification
API-01	Core endpoints: POST /signals (store normalized signal), POST /score-account (dynamic score), POST /generate-message (outreach), POST /sync-hubspot (CRM sync), plus analytics read APIs.
API-02	Every endpoint is versioned (e.g. /v1/...); breaking changes ship under a new version, never mutate an existing contract.
API-03	Every endpoint requires authentication and runs under least-privilege scopes; no anonymous access to contact/banking data.
API-04	Explicit request/response JSON schemas are published per endpoint; inputs are validated and rejected with a typed error on violation.
API-05	A stable error contract distinguishes retryable (timeout, 5xx, rate-limited) from permanent (validation, auth) failures for the FAIL layer.
API-06	Internal rate limits protect costly services (/generate-message, enrichment) from retry storms and runaway workflows.
API-07	Secrets/credentials are never passed in URLs or query strings; only in authenticated headers/body (privacy rule).
API-08	/score-account returns score + tier + the contributing factors (signal strength, regulatory pressure, reachability, relationship warmth) for explainability.
API-09	/generate-message accepts only grounded inputs from INT-SIG and rejects ungrounded personalization (hallucination gate).
API-10	n8n is the only orchestration caller; services do not call each other's databases, only their published APIs.

4.10 FAIL — Retry, Dead-Letter & Circuit-Breaker Architecture

The FSD specifies per-failure recovery, retry queues, dead-letter queues and circuit breakers. 4.10 consolidates these into one failure-recovery matrix so no failure is silent or unrecoverable.

Why #1: Why match recovery strategy to failure type rather than retry everything uniformly?

Because failures differ: a timeout is transient (retry), a validation error is permanent (don't retry), a CRM conflict needs human review (DLQ). Uniform retry either gives up too early or loops forever.

Why #2: Why dead-letter rather than drop or infinite-retry?

Because a dropped failure corrupts state silently and an infinite retry wedges the pipeline; a DLQ preserves the payload for inspection and replay while letting the pipeline proceed.

Why #3: Why exponential backoff specifically?

Because immediate aggressive retries against a struggling dependency worsen the outage (retry storm); backoff gives the dependency room to recover.

Why #4: Why circuit breakers on top of retries?

Because some failures (LinkedIn ban-risk) are made worse by retrying; the breaker stops the action entirely on early-warning signals rather than retrying into an irreversible ban.

Why #5: Why make every failure observable (no silent failure)?

Because the worst failure is the invisible one — it corrupts state and is discovered only when an account is mis-handled. Every failure must be logged, counted and surfaced.

Root cause: Failure handling is type-matched: transient → backoff retry, permanent → typed rejection, stateful-conflict → dead-letter, ban-risk → circuit-break — and every failure is observable, never silent.

Failure type	Recovery strategy	Visibility
API timeout	Exponential backoff retry	Logged; alert on repeated
AI generation failure	Retry with fallback prompt	Logged + QC flag
Email send failure	Queue retry	Reputation telemetry
CRM sync failure	Dead-letter queue	DLQ visible + alert
Enrichment failure	Deferred processing	Freshness flag to scoring
LinkedIn ban-risk signal	Circuit-breaker pause	Human-review flag + path-break

New rules

Rule ID	Specification
FAIL-01	Recovery strategy is selected by failure type per the matrix; the API error contract (API-05) supplies the classification.
FAIL-02	Transient failures (timeout, 5xx, rate-limited) use exponential backoff retry with a max-attempt cap.
FAIL-03	Permanent failures (validation, auth) are not retried; they return a typed error and are logged.
FAIL-04	Stateful-integration failures (CRM sync) dead-letter with full payload for inspection and replay (links to INT-CRM-03).
FAIL-05	AI generation failures retry with a fallback prompt then escalate to QC/human, never emit ungrounded output.
FAIL-06	Enrichment failures defer processing and flag contact freshness to scoring rather than blocking the pipeline.
FAIL-07	LinkedIn ban-risk and email reputation-floor breaches trigger circuit-breaker pauses (links INT-LIN-02 / INT-EML-04), not retries.
FAIL-08	No failure is silent: every failure is logged, counted, and alerts on repeated/threshold breach.
FAIL-09	Dead-letter and circuit-breaker states are human-reviewable and replayable; recovery actions are audited.

4.11 OBS — Observability, Security & Compliance

The platform handles enterprise contact data and banking context, and must remain aligned with PDPL, email compliance and LinkedIn platform policy. OBS specifies what is logged, secured and governed.

Why #1: Why log workflow execution, AI prompts/outputs, API failures and engagement events?

Because the engine makes autonomous decisions; without a full execution + reasoning trail, failures can't be debugged and decisions can't be explained or audited.

Why #2: Why log AI prompts and outputs specifically?

Because hallucination and metric-manipulation failure modes are only detectable by inspecting what the AI was asked and what it produced; the prompt/output log is the audit surface for AI governance.

Why #3: Why enforce role-based access + audit trails on this data?

Because enterprise contact and banking-context data is sensitive (PDPL); access must be least-privilege and every access auditable to demonstrate compliance.

Why #4: Why hold secrets in environment variables / encrypted credentials, never in code or URLs?

Because leaked credentials expose the CRM and enrichment data wholesale; secrets handling is the base of the security model and a hard privacy rule.

Why #5: Why make PDPL/compliance a continuous alignment requirement rather than a one-time check?

Because outreach scope, data retention and platform policies evolve; compliance must be a standing constraint the engine operates within, consistent with the Constraint Hierarchy's top tier (Legal/Compliance).

Root cause: Observability and compliance are a standing top-tier constraint: full execution + AI-reasoning logging, least-privilege audited access, encrypted secrets, and continuous PDPL/platform-policy alignment.

New rules

Rule ID	Specification
OBS-01	Mandatory logging: workflow execution, AI prompts, AI outputs, API failures, sequence activity, engagement events.
OBS-02	AI prompt/output logs are retained as the audit surface for hallucination and metric-manipulation detection.
OBS-03	Role-based access control + audit trails on all contact/banking data; access is least-privilege and recorded.
OBS-04	Secrets live in environment variables / encrypted credential stores; never in code, logs, or URLs.
OBS-05	Outreach data is restricted-access; CRM sync uses encrypted credentials (FSD security controls).
OBS-06	The platform operates within PDPL, email-compliance and LinkedIn platform-policy constraints as a continuous, top-tier (Legal/Compliance) requirement.
OBS-07	Compliance constraints sit at the top of the Constraint Hierarchy: no optimization, timing or capital rule may override them.

4.12 SECTION 4 RULE ROLL-UP & CROSS-LINKS

Domain	New rules	Key cross-links into Sections 1–3
INT-SIG	34	Scoring (Effective Opportunity), Market Memory, Timing windows, Scarcity (attention)
INT-ENR	7	Scarcity Economics (cost gating), reachability scoring
INT-LIN	7	SIG-PATH (warm-path matching, path-break), Timing (channel coordination)
INT-EML	8	Deliverability as strategic layer, contact state machine
INT-CRM	6	Human handoff (Principle 3), system-of-record integrity
INT-SLK	4	Timing as competitive weapon, grounded brief (Principle 5)
INT-ANL	4	Optimization loop, model governance
INT-ORC	7	Constraint Hierarchy (logic single-source-of-truth), idempotency
API	10	Explainability (Principle 3), hallucination gate (Module 0)
FAIL	9	Failure-mode-as-first-class-concern (Section 1 methodology)
OBS	7	Constraint Hierarchy top tier (Legal/Compliance), AI governance

SECTION 4 COMPLETE — WHAT IT ESTABLISHED

Section 4 specified the engine's entire external boundary: the INT-SIG intelligence acquisition engine (with SIG-PATH resolving the connection to Decimal against both Decimal's own network and generalized connectors), the ten supporting integrations, the five versioned service contracts, the type-matched failure architecture, and the compliance/observability constraint that sits above all of it.

~111 new rules, additive to the Section 1–3 base. Every integration was excavated via full 5-Whys and root-caused; every rule traces back to a failure it prevents.

ON THE HORIZON

Section 5 — Edge Cases & Failure Modes (cross-cutting scenarios that span modules + integrations), and the eventual consolidation of all section PDFs into the unified master specification.